# PYTHON_ACTIVITY_BASED_LEARNING


*Name – M. Sanjay Jonathan*
*Class – CSM 2*
*Roll.no – 26000960*


# *PROGRAM 1*

## *1.Problem Statement*

To develop a Python program to calculate electricity bills for multiple months based on the units of electricity consumed. The program applies different rates according to the consumption slabs, adds a fixed charge, provides a discount for low consumption, applies an additional charge of 5%, and identifies unusual increases in monthly bills. Finally, it calculates and displays the total electricity bill for all the months.

## *2. Objectives*

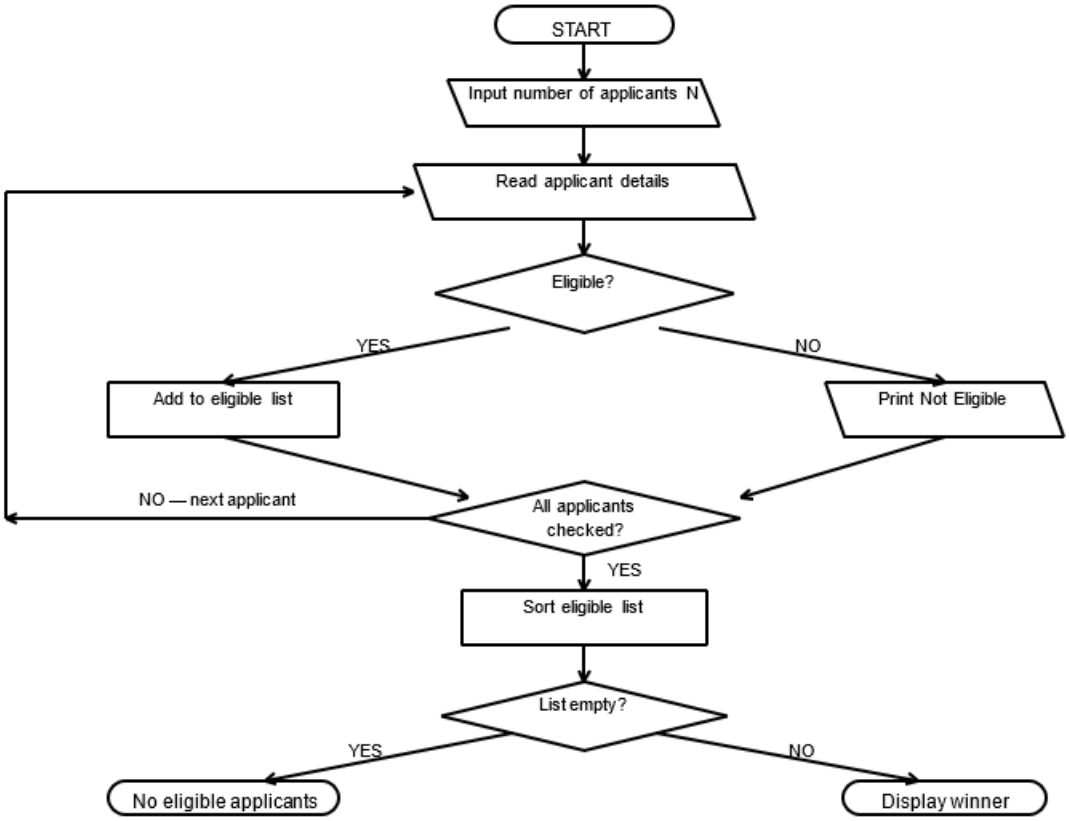
1. To accept the number of months for which electricity bills must be calculated.
2. To input the electricity units consumed during each month.
3. To calculate electricity charges using different consumption slabs.
4. To add a fixed charge of ₹50 to each monthly bill.
5. To provide a discount of ₹20 when electricity consumption is below 100 units.
6. To apply an additional charge of 5% to the bill after adjusting the fixed charge and discount.
7. To display the electricity bill for each month.
8. To identify unusual increases when a monthly bill exceeds 150% of the previous month's bill.
9. To calculate and display the total electricity bill for all the months.

## *3. Assumptions*

1. The number of months is entered as a positive integer.
2. Electricity consumption is entered as a non-negative numerical value.
3. The electricity charges are calculated using the following slabs:
    - Up to 100 units: ₹2 per unit.
    - 101 to 200 units: ₹3 per additional unit after the first 100 units.
    - 201 to 300 units: ₹5 per additional unit after the first 200 units.
    - Above 300 units: ₹7 per additional unit after the first 300 units.
4. A fixed charge of ₹50 is added to every monthly bill.
5. A discount of ₹20 is provided when consumption is below 100 units.

6. An additional charge of 5% is applied after adding the fixed charge and subtracting the discount, if applicable

## *FLOW-CHART*




## *Pseudo-Code*

```
START
Input N applicants
FOR each applicant:
    Read details     IF aggregate >= 75,
each subject >= 40,        attendance >=
75%, income <= 200000:         Add to
eligible list
    ELSE: Print "Not eligible"
Sort eligible list and display ranking
IF list not empty: Display winner
ELSE: Print "No eligible applicants"
STOP
```

# Algorithm

1. Start.
2. Read the number of applicants.
3. Enter each student's details.
4. Check the eligibility conditions.
5. Store the eligible students.
6. Sort eligible students according to their ranking.
7. Display the ranking list.
8. Award the scholarship to the top-ranked student.
9. If no students are eligible, display a message.
10. Stop.

## ***Code***

```
n = int(input("Enter number of applicants: "))
students = []

for i in range(n):
    name = input("Enter student name: ")
    marks = float(input("Enter aggregate marks: "))
    s1 = float(input("Enter subject 1 marks: "))
    s2 = float(input("Enter subject 2 marks: "))
    attendance = float(input("Enter attendance percentage: "))
    income = float(input("Enter annual family income: "))

    if marks >= 75 and s1 >= 40 and s2 >= 40 and attendance >= 75 and income <= 200000:
        students.append([name, marks, attendance, income])
        print(name, "is eligible")
    else:
        print(name, "is not eligible")

students.sort(key=lambda x: (x[1], x[2], -x[3]), reverse=True)

print("\nScholarship Ranking:")
for i in range(len(students)):
    print(i + 1, students[i][0], students[i][1])

if len(students) > 0:
    print("\nScholarship awarded to:", students[0][0])
else:
    print("\nNo eligible applicants")
```

## ***4. Output***

Sample Output:
Number of months: 3

Units consumed: 80
Bill: 198.0
Units consumed: 150
Bill: 522.5
Units consumed: 350
Bill: 1470.0
Unusual increase in month 2
Unusual increase in month 3
Total bill: 2190.5

## *5. Conclusion*

The Python program successfully calculates electricity bills based on monthly consumption using different tariff slabs. It applies fixed charges, discounts, and additional charges according to the specified conditions. The program also identifies unusual increases in monthly bills and calculates the total electricity expense for all the months. This helps users understand their electricity expenses and monitor significant changes in consumption costs.

# *PROGRAM 2*

**1. Problem Statement**

To develop a Python program to calculate electricity bills for multiple months based on the units of electricity consumed. The program applies different rates according to the consumption slabs, adds a fixed charge, provides a discount for low consumption, applies an additional charge of 5%, and identifies unusual increases in monthly bills. Finally, it calculates and displays the total electricity bill for all the months.

**2. Objectives**

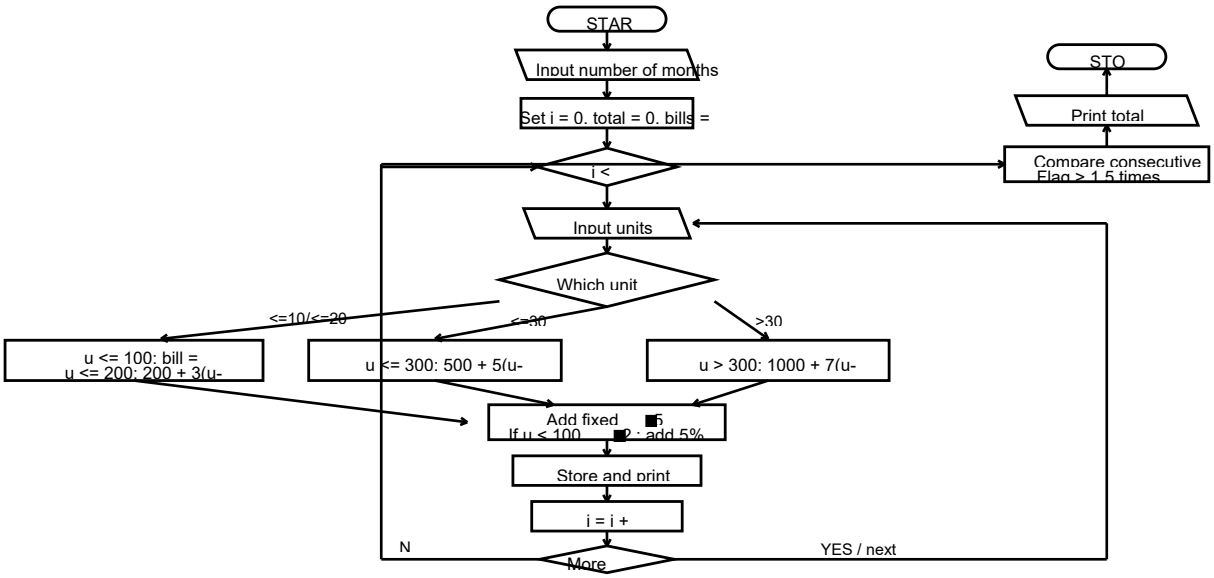
10. To accept the number of months for which electricity bills must be calculated.
11. To input the electricity units consumed during each month.
12. To calculate electricity charges using different consumption slabs.
13. To add a fixed charge of ₹50 to each monthly bill.
14. To provide a discount of ₹20 when electricity consumption is below 100 units.
15. To apply an additional charge of 5% to the bill after adjusting the fixed charge and discount.
16. To display the electricity bill for each month.
17. To identify unusual increases when a monthly bill exceeds 150% of the previous month's bill.
18. To calculate and display the total electricity bill for all the months.

**3. Assumptions**

1. The number of months is entered as a positive integer.
2. Electricity consumption is entered as a non-negative numerical value.
3. The electricity charges are calculated using the following slabs:
    - Up to 100 units: ₹2 per unit.
    - 101 to 200 units: ₹3 per additional unit after the first 100 units.
    - 201 to 300 units: ₹5 per additional unit after the first 200 units.
    - Above 300 units: ₹7 per additional unit after the first 300 units.
4. A fixed charge of ₹50 is added to every monthly bill.
5. A discount of ₹20 is provided when consumption is below 100 units.
6. An additional charge of 5% is applied after adding the fixed charge and subtracting the discount, if applicable.

7. An unusual increase is reported when the current month's bill is more than 1.5 times the previous month's bill.
8. The total bill is calculated by adding the bills for all the months.

## *FLOW-CHART*




## *Pseudo-Code*

START

Input n months; create empty bill list FOR each month:

Input units u  Calculate bill using slabs:

u<=100: 2u; <=200: 200+3(u-100)  <=300: 500+5(u-200); >300: 1000+7(u-300)

Add ■50; if u<100 subtract ■20; add 5% tax

Store and print bill

Check consecutive bills; flag if current > 1.5 × previous

Print total bill

STOP

## *Algorithm*

1. Start.
2. Enter the number of months.
3. Enter the electricity units consumed for each month.
4. Calculate the bill based on the units consumed.
5. Add the fixed charge and applicable adjustments.
6. Calculate the final bill after adding 5% tax.
7. Display the monthly bill.
8. Compare consecutive monthly bills and detect unusual increases.
9. Calculate and display the total electricity bill.
10. Stop.

## ***Code***

```
n = int(input("Enter number of patients: "))

for i in range(n):
    name = input("Enter patient name: ")
    consultation = float(input("Enter consultation charges: "))
    diagnostic = float(input("Enter diagnostic charges: "))
    treatment = float(input("Enter treatment charges: "))

    total = consultation + diagnostic + treatment

    if consultation < 0 or diagnostic < 0 or treatment < 0:
        print("Invalid billing charges")
        continue

    if total >= 10000:
        concession = total * 0.10
    elif total >= 5000:
        concession = total * 0.05
    else:
        concession = 0

    final_bill = total - concession

    if final_bill <= 0:
        print("Invalid final bill")
    elif final_bill <= 3000:
        category = "Low Payment"
    elif final_bill <= 10000:
        category = "Medium Payment"
    else:
        category = "High Payment"

    print("\nPatient:", name)
    print("Total Charges:", total)
```

```
print("Concession:", concession)
print("Final Bill:", final_bill)
print("Payment Category:", category)
```

**4. Output**

Sample Output:
Number of months: 3
Units consumed: 80
Bill: 198.0
Units consumed: 150
Bill: 522.5
Units consumed: 350
Bill: 1470.0
Unusual increase in month 2
Unusual increase in month 3
Total bill: 2190.5

**5. Conclusion**

The Python program successfully calculates electricity bills based on monthly consumption using different tariff slabs. It applies fixed charges, discounts, and additional charges according to the specified conditions. The program also identifies unusual increases in monthly bills and calculates the total electricity expense for all the months. This helps users understand their electricity expenses and monitor significant changes in consumption costs.

# *PROGRAM 3*

**1. Problem Statement**

To develop a Python program to calculate electricity bills for multiple months based on the units of electricity consumed. The program applies different rates according to the consumption slabs, adds a fixed charge, provides a discount for low consumption, applies an additional charge of 5%, and identifies unusual increases in monthly bills. Finally, it calculates and displays the total electricity bill for all the months.

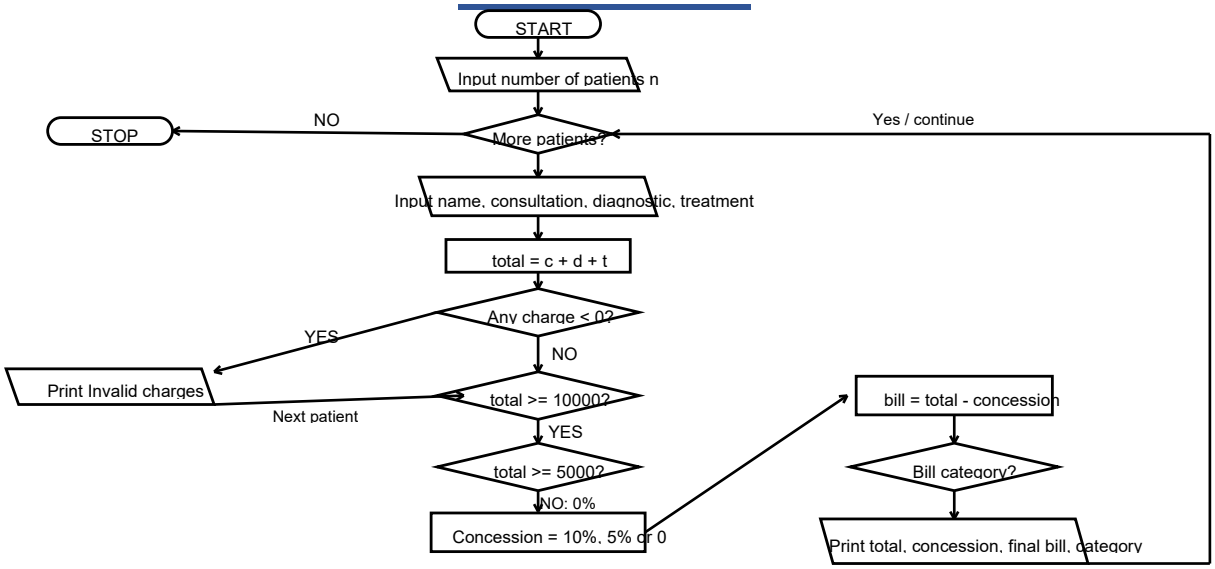
**2. Objectives**

19. To accept the number of months for which electricity bills must be calculated.
20. To input the electricity units consumed during each month.
21. To calculate electricity charges using different consumption slabs.
22. To add a fixed charge of ₹50 to each monthly bill.
23. To provide a discount of ₹20 when electricity consumption is below 100 units.
24. To apply an additional charge of 5% to the bill after adjusting the fixed charge and discount.
25. To display the electricity bill for each month.
26. To identify unusual increases when a monthly bill exceeds 150% of the previous month's bill.
27. To calculate and display the total electricity bill for all the months.

**3. Assumptions**

1. The number of months is entered as a positive integer.
2. Electricity consumption is entered as a non-negative numerical value.
3. The electricity charges are calculated using the following slabs:
    - Up to 100 units: ₹2 per unit.
    - 101 to 200 units: ₹3 per additional unit after the first 100 units.
    - 201 to 300 units: ₹5 per additional unit after the first 200 units.
    - Above 300 units: ₹7 per additional unit after the first 300 units.
4. A fixed charge of ₹50 is added to every monthly bill.
5. A discount of ₹20 is provided when consumption is below 100 units.
6. An additional charge of 5% is applied after adding the fixed charge and subtracting the discount, if applicable.
7. An unusual increase is reported when the current month's bill is more than 1.5 times the previous month's bill.
8. The total bill is calculated by adding the bills for all the months.

## *FLOW-CHART*




## *Pseudo-Code*

START

Input number of patients n FOR each patient: Input name, consultation, diagnostic, treatment charges total = consultation + diagnostic + treatment IF any charge < 0: Print "Invalid charges"; continue

IF total >= 10000: concession = 10% of total

ELSE IF total >= 5000: concession = 5% of total ELSE: concession = 0 bill = total - concession Categorize bill: <=3000 Low; <=10000 Medium; else High

Print total, concession, final bill, category

STOP

## *Algorithm*

1. Start.
2. Enter the number of patients.
3. Enter each patient's name and charges.
4. Calculate the total charges.
5. Check whether the charges are valid.
6. Calculate the concession based on the total charges.
7. Calculate the final bill.
8. Classify the payment as low, medium, or high.
9. Display the patient's bill details.
10. Repeat for all patients.
11. Stop.

## *Code*

```
n = int(input("Enter number of patients: "))

for i in range(n):
    name = input("Enter patient name: ")
    consultation = float(input("Enter consultation charges: "))
    diagnostic = float(input("Enter diagnostic charges: "))
    treatment = float(input("Enter treatment charges: "))

    total = consultation + diagnostic + treatment

    if consultation < 0 or diagnostic < 0 or treatment < 0:
        print("Invalid billing charges")
        continue
```

```
    if total >= 10000:
        concession = total * 0.10
    elif total >= 5000:
        concession = total * 0.05
    else:
        concession = 0

    final_bill = total - concession

    if final_bill <= 0:
        print("Invalid final bill")
    elif final_bill <= 3000:
        category = "Low Payment"
    elif final_bill <= 10000:
        category = "Medium Payment"
    else:
        category = "High Payment"

    print("\nPatient:", name)
    print("Total Charges:", total)
    print("Concession:", concession)
    print("Final Bill:", final_bill)
    print("Payment Category:", category)
```

**4. Output**

Sample Output:

Number of months: 3
Units consumed: 80
Bill: 198.0
Units consumed: 150
Bill: 522.5
Units consumed: 350
Bill: 1470.0
Unusual increase in month 2
Unusual increase in month 3

Total bill: 2190.5

**5. Conclusion**

The Python program successfully calculates electricity bills based on monthly consumption using different tariff slabs. It applies fixed charges, discounts, and additional charges according to the specified conditions. The program also identifies unusual increases in monthly bills and calculates the total electricity expense for all the months. This helps users understand their electricity expenses and monitor significant changes in consumption costs.

# *PROGRAM 4*

**1. Problem Statement**

To develop a Python program to calculate the gross salary, salary deduction, and net salary of multiple employees based on their basic pay, allowances, overtime, and incentives. The program validates salary components, applies deductions according to the gross salary slabs, and displays the gross salary, deduction amount, and final net salary for each employee.

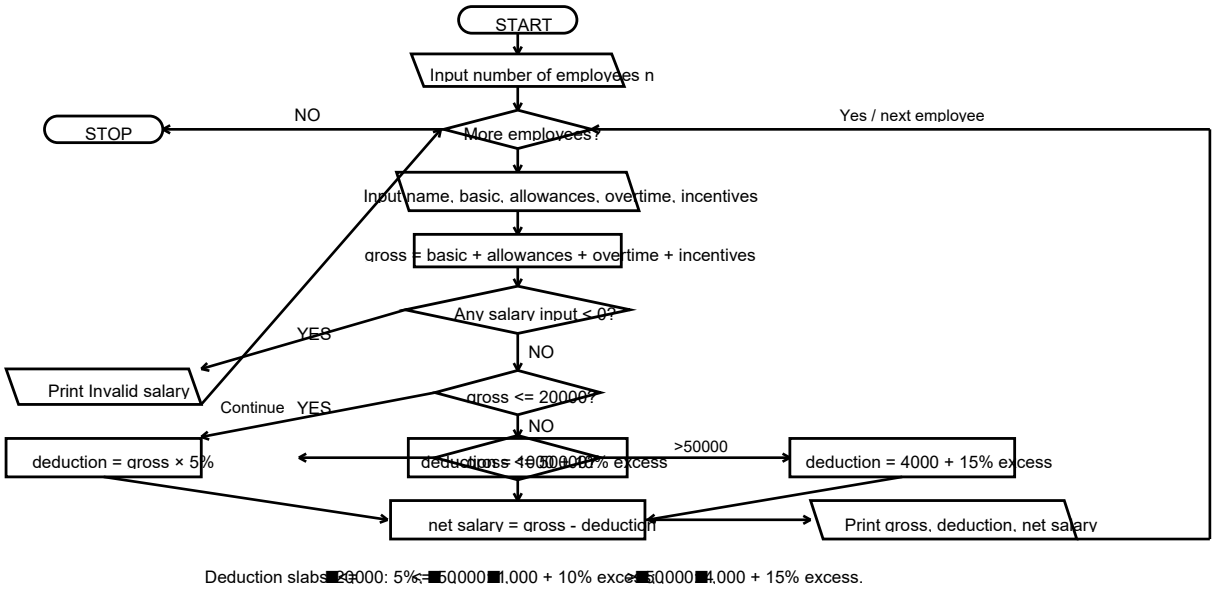
**2. Objectives**

1. To accept the number of employees and their salary details.
2. To input the basic pay, allowances, overtime, and incentives for each employee.
3. To validate salary components and identify negative values as invalid.
4. To calculate the gross salary by adding basic pay, allowances, overtime, and incentives.
5. To calculate salary deductions based on the gross salary slabs.
6. To apply a deduction of 5% for gross salaries up to ₹20,000.
7. To calculate deductions using the specified slab formula for gross salaries above ₹20,000.
8. To calculate the net salary by subtracting the deduction from the gross salary.
9. To display the gross salary, deduction amount, and net salary for each valid employee.

**3. Assumptions**

1. The number of employees is entered as a non-negative integer.
2. Each employee has a valid name.
3. Basic pay, allowances, overtime, and incentives are entered as numerical values in Indian rupees.
4. Negative values in any salary component are considered invalid, and the program skips further calculations for that employee.
5. Gross salary is calculated by adding basic pay, allowances, overtime, and incentives.
6. A deduction of 5% of gross salary is applied when the gross salary is ₹20,000 or less.
7. For gross salaries above ₹20,000 and up to ₹50,000, the deduction is ₹1,000 plus 10% of the amount exceeding ₹20,000.
8. For gross salaries above ₹50,000, the deduction is ₹4,000 plus 15% of the amount exceeding ₹50,000.
9. Net salary is calculated by subtracting the deduction from the gross salary.
10. Salary deductions and net salaries are displayed rounded to two decimal places.

# FLOW-CHART




# Pseudo-Code

START

Input number of employees n FOR each employee: Input name, basic pay, allowances, overtime, incentives gross = basic + allowances + overtime + incentives IF any input < 0: Print "Invalid salary"; continue

IF gross <= 20000: deduction = 5% of gross

ELSE IF gross <= 50000: deduction = 1000 + 10% of (gross-20000) ELSE: deduction = 4000 + 15% of (gross-50000) net = gross - deduction Print name, gross, deduction, net

STOP

# Algorithm

1. Start.
2. Enter the number of employees.
3. Enter each employee's name and salary details.
4. Check whether the salary details are valid.
5. Calculate the gross salary.

6. Calculate the deductions based on the gross salary.
7. Calculate the net salary.
8. Display the salary statement.
9. Repeat for all employees.
10. Stop.

## ***Code***

```
n = int(input("Enter number of employees: "))

for i in range(n):
    name = input("Enter employee name: ")
    basic = float(input("Enter basic pay: "))
    allowance = float(input("Enter allowances: "))
    overtime = float(input("Enter overtime pay: "))
    incentive = float(input("Enter incentives: "))

    if basic < 0 or allowance < 0 or overtime < 0 or incentive < 0:
        print("Invalid salary details")
        continue

    gross = basic + allowance + overtime + incentive

    if gross <= 20000:
        deduction = gross * 0.05
    elif gross <= 50000:
        deduction = 1000 + (gross - 20000) * 0.10
    else:
        deduction = 4000 + (gross - 50000) * 0.15

    net = gross - deduction

    print("\n----- Salary Statement -----")
    print("Employee Name:", name)
    print("Basic Pay:", basic)
```

```
print("Allowances:", allowance)
print("Overtime:", overtime)
print("Incentives:", incentive)
print("Gross Salary:", round(gross, 2))
print("Deductions:", round(deduction, 2))
print("Net Salary:", round(net, 2))
```

**4. Output**

Sample Output:
Number of employees: 3
Employee name: Ravi
Basic pay: 15000
Allowances: 3000
Overtime: 1000
Incentives: 1000
Employee: Ravi Gross: 20000.0
Deduction: 1000.0 Net Salary: 19000.0
Employee name: Sita
Basic pay: 25000
Allowances: 5000
Overtime: 2000
Incentives: 3000
Employee: Sita Gross: 35000.0
Deduction: 2500.0 Net Salary: 32500.0
Employee name: Kumar
Basic pay: 40000
Allowances: 8000
Overtime: 3000
Incentives: 4000
Employee: Kumar Gross: 55000.0
Deduction: 4750.0 Net Salary: 50250.0

**5. Conclusion**

The Python program successfully calculates employee salaries by considering basic pay, allowances, overtime, and incentives. It validates salary components, calculates gross salary, applies deductions according to the specified salary slabs, and determines the final net salary. This program helps simplify employee payroll calculations and improves the accuracy and efficiency of salary processing.

# PROGRAM 5

**1. Problem Statement**

To develop a Python program to compare multiple mobile recharge plans based on data usage, call minutes, price, validity, and the user's budget. The program calculates the total cost of each plan, including additional charges for excess data usage and call minutes. It then identifies the most economical plan that satisfies the user's budget and minimum validity requirements.

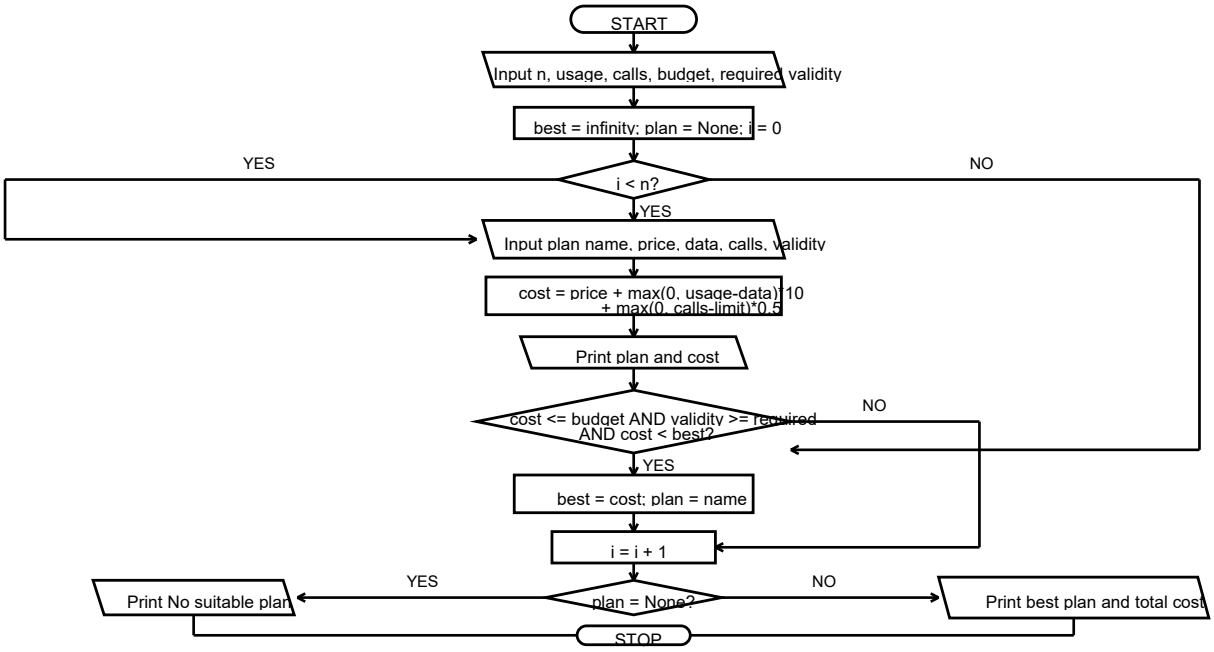
**2. Objectives**

1. To accept the user's data usage, call minutes, budget, and required validity.
2. To input the details of multiple mobile recharge plans.
3. To calculate the total cost of each plan based on its price and additional usage charges.
4. To charge ₹10 for each unit of data usage exceeding the plan's data limit.
5. To charge ₹0.50 for each call minute exceeding the plan's call limit.
6. To display the total calculated cost of each plan.
7. To select the lowest-cost plan that satisfies the user's budget and minimum validity requirements.
8. To display the name and total cost of the best suitable plan.
9. To display a message when no plan satisfies the specified requirements.

**3. Assumptions**

1. The number of plans is entered as a positive integer.
2. Data usage, call minutes, budget, and plan prices are entered as non-negative numerical values.
3. The required validity and plan validity are entered as integers representing days.
4. Each plan has a name, price, data limit, call limit, and validity period.
5. Additional data usage is charged at ₹10 per unit above the plan's data limit.
6. Additional call usage is charged at ₹0.50 per minute above the plan's call limit.
7. No additional charges are applied when data usage and call minutes remain within the plan limits.
8. A plan is considered suitable only when its total calculated cost does not exceed the budget and its validity is equal to or greater than the required validity.
9. Among all suitable plans, the plan with the lowest total calculated cost is selected.
10. If no plan satisfies the budget and validity requirements, the program displays "No suitable plan".

## ***FLOW-CHART***




# *Pseudo-Code*

1. Start.
2. Enter the number of mobile plans.
3. Enter monthly data usage, call minutes, budget, and required validity.
4. Enter the details of each mobile plan.
5. Calculate the extra charges for data and calls.
6. Calculate the total cost of each plan.
7. Check whether the plan satisfies the budget and validity requirements.
8. Select the cheapest eligible plan.
9. Display the best plan and its total cost.
10. If no plan is eligible, display a message.
11. Stop.

# *Algorithm*

12. Start.
13. Enter the number of mobile plans.
14. Enter monthly data usage, call minutes, budget, and required validity.
15. Enter the details of each mobile plan.
16. Calculate the extra charges for data and calls.
17. Calculate the total cost of each plan.
18. Check whether the plan satisfies the budget and validity requirements.
19. Select the cheapest eligible plan.
20. Display the best plan and its total cost.
21. If no plan is eligible, display a message.
22. Stop.

# Code

```
n = int(input("Enter number of mobile plans: "))
usage = float(input("Enter monthly data usage in GB: "))
calls = float(input("Enter monthly call minutes: "))
budget = float(input("Enter your budget: "))
validity = int(input("Enter required validity in days: "))

best_cost = float("inf")
best_plan = "None"

for i in range(n):
    name = input("Enter plan name: ")
    price = float(input("Enter plan price: "))
    data = float(input("Enter data limit in GB: "))
    minutes = float(input("Enter call limit in minutes: "))
    days = int(input("Enter validity in days: "))
    extra = max(0, usage - data) * 10
    extra += max(0, calls - minutes) * 0.5
    total = price + extra

    print("Plan:", name, "Total Cost:", round(total, 2))

    if total <= budget and days >= validity:
        if total < best_cost:
            best_cost = total
            best_plan = name

print("\nMost Economical Eligible Plan:", best_plan)
if best_plan != "None":
    print("Total Cost:", round(best_cost, 2))
else:
    print("No plan satisfies the given constraints")
```

**4. Output**

Sample Output:
Number of plans: 3
Data usage: 2
Call minutes: 100
Budget: 300
Required validity: 28
Plan name: Basic
Price: 199
Data limit: 1
Call limit: 80
Validity: 28
Basic Total cost: 219.0
Plan name: Standard

Price: 249
Data limit: 2
Call limit: 150
Validity: 56
Standard Total cost: 249.0
Plan name: Premium
Price: 399
Data limit: 3
Call limit: 200
Validity: 84
Premium Total cost: 399.0
Best plan: Standard
Total cost: 249.0

**5. Conclusion**
The Python program successfully compares mobile recharge plans by considering data usage, call minutes, additional charges, budget, and validity requirements. It calculates the total cost of each plan and selects the most economical option that meets the user's requirements. If no suitable plan is available, the program displays an appropriate message. This program helps users compare recharge plans and make cost-effective decisions.

# *PROGRAM 6*

**1. Problem Statement**
To develop a Python program to calculate the final railway ticket fare for multiple passengers based on their travel distance, age, and selected travel class. The program applies different fare rates for Sleeper, AC, and General classes, provides concessions for children and senior citizens, adjusts the fare for ticket cancellations, and displays the final fare for each passenger.

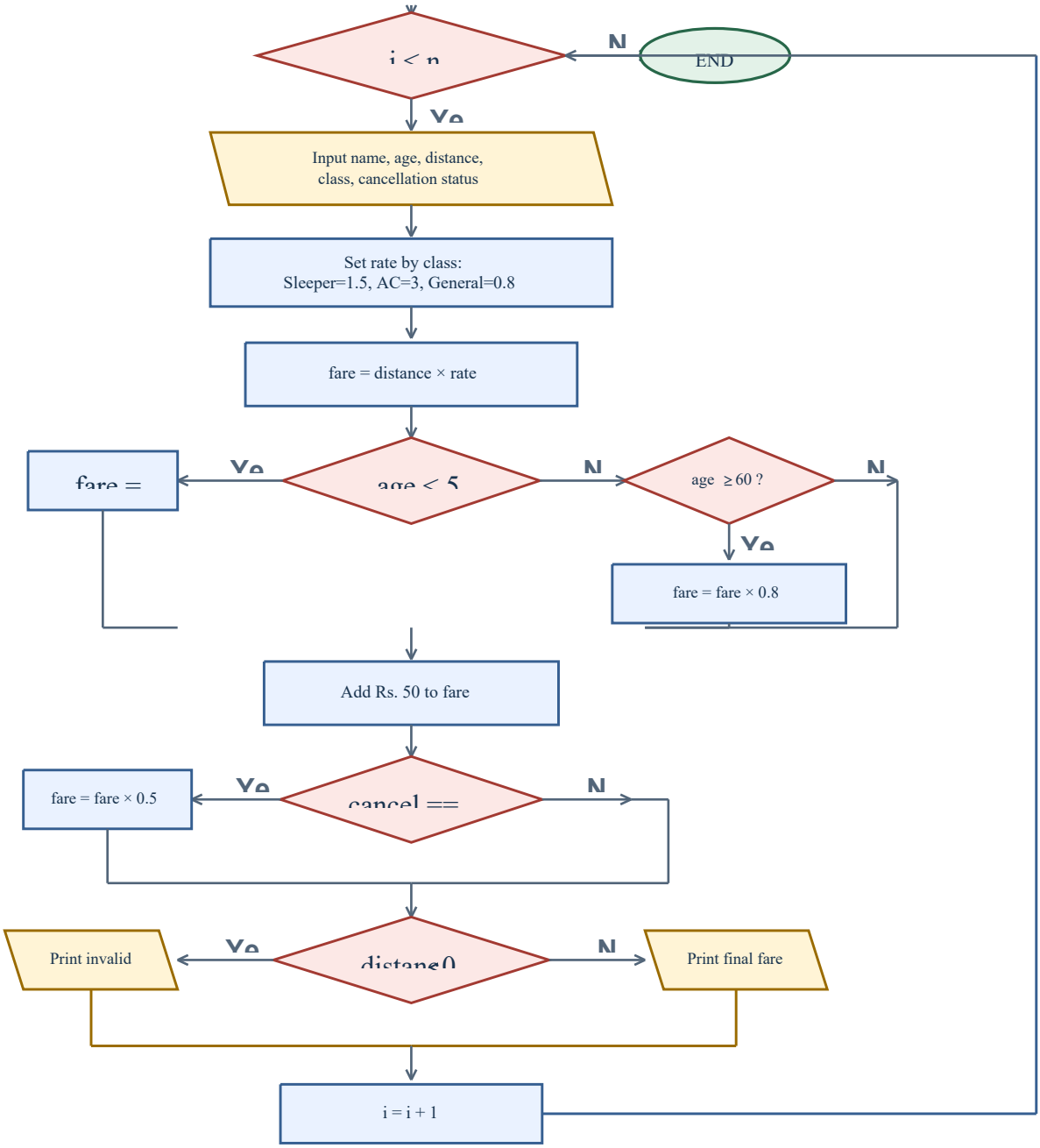
**2. Objectives**
1. To accept the number of passengers and their travel details.
2. To input each passenger's name, age, travel distance, and selected travel class.
3. To calculate the basic fare based on the distance travelled and the selected class.
4. To provide free travel fare for children below 5 years of age.
5. To provide a 20% fare concession for senior citizens aged 60 years or above.
6. To add a fixed charge of ₹50 to the calculated fare.
7. To reduce the fare by 50% when a ticket is cancelled.
8. To identify invalid travel distances.
9. To display the final fare for each passenger after applying the applicable conditions.

**3. Assumptions**
1. The number of passengers is entered as a non-negative integer.
2. Each passenger provides a valid name and a non-negative age.
3. The travel distance is entered in kilometres.
4. The travel class is selected using the following options:
    - 1 for Sleeper class at ₹1.50 per kilometre.
    - 2 for AC class at ₹3.00 per kilometre.

- 3 for General class at ₹0.80 per kilometre.

5. Children below 5 years of age receive a basic fare exemption.
6. Senior citizens aged 60 years or above receive a 20% concession on the calculated distance-based fare.
7. A fixed charge of ₹50 is added after applying the age-based concession.
8. If the ticket is cancelled by entering "yes", the final fare is reduced by 50%.
9. A travel distance of zero or less is considered invalid.
10. The final fare is displayed rounded to two decimal places.
11. The program assumes that the cancellation response is entered as "yes" or "no" in lowercase.

## ***FLOW-CHART***



# *Pseudo-Code*

START
Input number of passengers n
Repeat for each passenger:
  Input name, age, distance,
  class, and cancellation status
  Set rate: Sleeper=1.5, AC=3,
  otherwise General=0.8
  fare = distance × rate
  If age < 5, fare = 0
  Else if age ≥ 60, fare = fare × 0.8
  Add Rs. 50 to fare
  If cancelled, fare = fare × 0.5
  If distance ≤ 0, print invalid
  Else print final fare
END

# *Algorithm*

1. Start.
2. Enter the number of passengers.
3. Enter each passenger's name, age, distance, travel class, and cancellation choice.
4. Calculate the fare based on the travel class.
5. Apply the age discount, if applicable.
6. Add the service charge.
7. Apply the cancellation charge, if applicable.
8. Check whether the details are valid.
9. Display the passenger details and final fare.
10. Repeat for all passengers.
11. Stop.

# *Code*

```
n = int(input("Enter number of passengers: "))

for i in range(n):
```

```
name = input("Enter passenger name: ")
age = int(input("Enter passenger age: "))
distance = float(input("Enter travel distance in km: "))
print("1. Sleeper  2. AC  3. General")
cls = int(input("Choose travel class: "))
cancel = input("Cancel ticket? (yes/no): ")

if cls == 1:
    rate = 1.5
elif cls == 2:
    rate = 3.0
else:
    rate = 0.8

fare = distance * rate

if age < 5:
    fare = 0
elif age >= 60:
    fare = fare * 0.8

fare += 50

if cancel == "yes":
    fare = fare * 0.5

if fare < 0 or distance <= 0:
    print("Invalid details")
    continue

print("\nPassenger:", name)
print("Distance:", distance, "km")
print("Travel Class:", cls)
print("Final Fare: Rs.", round(fare, 2))
```

**4. Output**

Sample Output:
Number of passengers: 3
Passenger name: Ravi
Age: 30
Distance in km: 200
1.Sleeper 2.AC 3.General: 1
Cancel ticket? yes/no: no
Ravi Final Fare: Rs. 350.0
Passenger name: Sita
Age: 65
Distance in km: 300
1.Sleeper 2.AC 3.General: 2
Cancel ticket? yes/no: no
Sita Final Fare: Rs. 770.0
Passenger name: Kumar
Age: 25
Distance in km: 100
1.Sleeper 2.AC 3.General: 3
Cancel ticket? yes/no: yes
Kumar Final Fare: Rs. 65.0

**5. Conclusion**
The Python program successfully calculates railway ticket fares based on travel distance, passenger age, and selected travel class. It applies age-based concessions, adds a fixed charge, and adjusts the fare when a ticket is cancelled. It also identifies invalid travel distances and displays the final fare for each passenger. This program helps simplify railway fare calculations and provides a systematic approach to passenger ticket management.

# *PROGRAM 7*

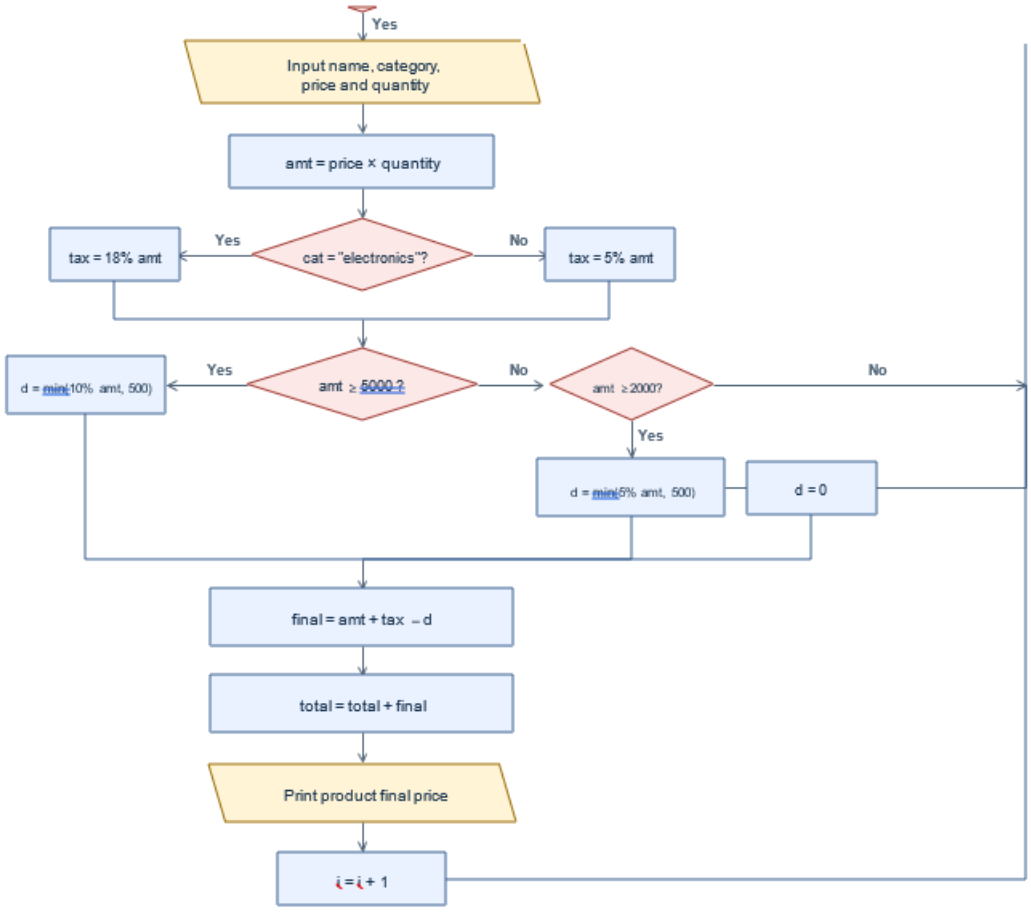
**1. Problem Statement**
To develop a Python program to calculate the final prices of multiple products based on their individual prices, quantities, categories, applicable taxes, and discounts. The program calculates the total amount for each product, applies different tax rates according to the product category, provides discounts based on the purchase amount, and generates the final invoice amount for all products.

**2. Objectives**
1. To accept the number of products and their details.
2. To input the product name, category, price, and quantity.
3. To calculate the total amount for each product by multiplying its price by its quantity.
4. To apply an 18% tax rate to electronics products.
5. To apply a 5% tax rate to products belonging to other categories.
6. To provide a 10% discount for product amounts of ₹5,000 or more, subject to a maximum discount of ₹500.
7. To provide a 5% discount for product amounts of ₹2,000 or more but less than ₹5,000, subject to a maximum discount of ₹500.
8. To calculate the final price of each product after adding tax and subtracting the discount.
9. To calculate and display the total invoice amount for all products.

**3. Assumptions**
1. The number of products is entered as a non-negative integer.
2. Each product has a valid name and category.
3. The product price is entered as a non-negative numerical value in Indian rupees.
4. The quantity of each product is entered as a non-negative integer.
5. The total amount of a product is calculated by multiplying its price by its quantity.
6. Electronics products are identified by entering the category as "electronics" in lowercase and are charged 18% tax.
7. Products belonging to all other categories are charged 5% tax.
8. A 10% discount is applied when the product amount is ₹5,000 or more, with the discount limited to ₹500.
9. A 5% discount is applied when the product amount is ₹2,000 or more but less than ₹5,000, with the discount limited to ₹500.
10. No discount is provided when the product amount is below ₹2,000.
11. The final price is calculated by adding tax to the product amount and subtracting the applicable discount.

12. The total invoice amount is calculated by adding the final prices of all products.

## ***FLOW-CHART***



# *Pseudo-Code*

```
START
Input number of products n
Set total = 0
Repeat n times:
  Input name, category, price,
  and quantity
  amt = price × quantity
  If electronics: tax = 18% of amt
  Else: tax = 5% of amt
  If amt ≥ 5000: discount = min(10% of amt, 500)
  Else if amt ≥ 2000: discount = min(5% of amt, 500)
  Else: discount = 0
  final = amt + tax − discount
  total = total + final
  Print product final price
Print total invoice
END
```

## *Algorithm*

1. Start.
2. Enter the number of products.
3. Enter each product's name, category, price, and quantity.
4. Calculate the amount.
5. Calculate the tax based on the category.
6. Calculate the discount based on the amount.
7. Limit the discount to ₹500.
8. Calculate the final price.
9. Add each final price to the total invoice amount.
10. Display the product details and total invoice amount.
11. Stop.

# ***Code***

```
n = int(input("Enter number of products: "))
total = 0
max_discount = 500

for i in range(n):
    name = input("Enter product name: ")
    category = input("Enter category (food/clothing/electronics): ")
    price = float(input("Enter price: "))
    quantity = int(input("Enter quantity: "))

    amount = price * quantity

    if category == "food":
        tax = amount * 0.05
    elif category == "clothing":
        tax = amount * 0.05
    else:
        tax = amount * 0.18

    discount = 0
    if amount >= 5000:
        discount = amount * 0.10
    elif amount >= 2000:
        discount = amount * 0.05

    discount = min(discount, max_discount)
    final = amount + tax - discount
    total += final

    print("\nProduct:", name)
    print("Amount:", amount, "Tax:", round(tax, 2))
    print("Discount:", round(discount, 2))
    print("Final Price:", round(final, 2))

print("\nTotal Invoice Amount: Rs.", round(total, 2))
```

**4. Output**

Sample Output:
Number of products: 3
Product name: Laptop
Category: electronics
Price: 40000
Quantity: 1
Laptop Final Price: 46700.0
Product name: Headphones
Category: electronics
Price: 3000
Quantity: 1
Headphones Final Price: 3440.0
Product name: Notebook
Category: stationery
Price: 100
Quantity: 5
Notebook Final Price: 525.0
Total Invoice: Rs. 50665.0

**5. Conclusion**
The Python program successfully calculates the final prices of multiple products by considering their prices, quantities, categories, applicable taxes, and discounts. It applies different tax rates and discount rules to determine the final amount payable for each product. It also calculates the total invoice amount for all products, helping to simplify billing and improve the accuracy of invoice calculations.

# *PROGRAM 8*

## 1. Problem Statement

To develop a Python program to calculate the total production cost, revenue, profit or loss, and profit percentage for multiple products based on material cost, labour cost, wastage cost, machine cost, selling price, and quantity. The program validates the input values, compares the calculated profit percentage with the target profit percentage, and determines whether the target has been achieved for each product.

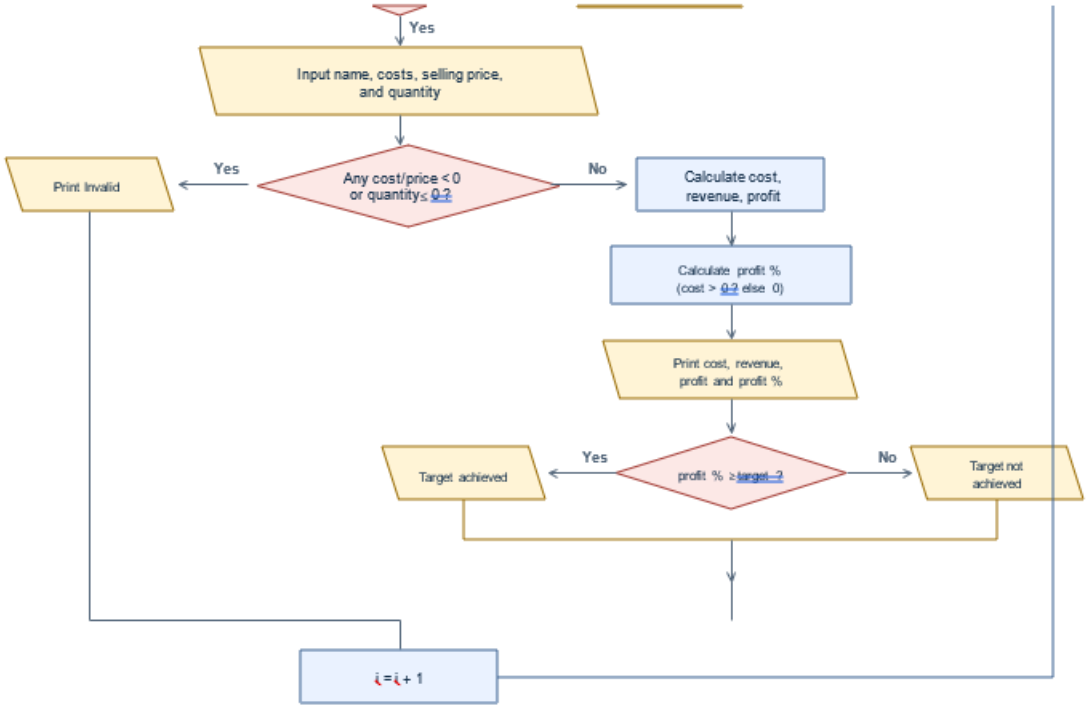
## 2. Objectives

1. To accept the number of products and the target profit percentage.
2. To input the product name, material cost, labour cost, wastage cost, machine cost, selling price, and quantity.
3. To validate the input values and identify invalid costs, selling prices, or quantities.
4. To calculate the total production cost for each product based on its quantity.
5. To calculate the total revenue by multiplying the selling price by the quantity.
6. To determine the profit or loss by calculating the difference between revenue and total production cost.
7. To calculate the profit percentage for each product.
8. To compare the calculated profit percentage with the target profit percentage.
9. To display whether the target profit has been achieved for each valid product.

## 3. Assumptions

1. The number of products is entered as a non-negative integer.
2. The target profit percentage is entered as a numerical value.
3. Each product has a valid name.
4. Material cost, labour cost, wastage cost, machine cost, and selling price are entered as non-negative numerical values.
5. The quantity of each product must be a positive integer.
6. Negative values for costs or selling price, and quantities less than or equal to zero, are considered invalid inputs.
7. Total production cost is calculated by adding the material, labour, wastage, and machine costs and multiplying the result by the quantity.
8. Total revenue is calculated by multiplying the selling price by the quantity.
9. Profit or loss is calculated by subtracting total production cost from total revenue.
10. Profit percentage is calculated as (Profit or Loss ÷ Total Production Cost) × 100 when the total production cost is greater than zero. Otherwise, the profit percentage is considered zero.
11. The target is achieved when the calculated profit percentage is greater than or equal to the target profit percentage

# FLOW-CHART




# Pseudo-Code

START

Input n and target profit %

Repeat for each product:

Input name, material, labour,

wastage, machine cost, selling

price, and quantity

If any cost/price < 0 or q ≤ 0:

Print Invalid input; continue

cost = (m + l + w + mc) × q

revenue = selling price × q

profit = revenue − cost

If cost > 0: profit% = profit/cost × 100

Else: profit% = 0

Print cost, revenue, profit, profit%

If profit% ≥ target: print Target achieved

Else: print Target not achieved

END

## *Algorithm*

1. Start.
2. Enter the number of products and target profit percentage.
3. Enter the details of each product.
4. Check whether the inputs are valid.
5. Calculate the total production cost.
6. Calculate the total revenue.
7. Calculate the profit or loss.
8. Calculate the profit percentage.
9. Check whether the target profit percentage is achieved.
10. Display the results.
11. Repeat for all products.
12. Stop.

## ***Code***

```
n = int(input("Enter number of products: "))
target = float(input("Enter target profit percentage: "))

for i in range(n):
    name = input("Enter product name: ")
    material = float(input("Enter material cost: "))
    labour = float(input("Enter labour charges: "))
    wastage = float(input("Enter wastage cost: "))
    machine = float(input("Enter machine cost: "))
    selling = float(input("Enter selling price: "))
    quantity = int(input("Enter quantity produced: "))

    if material < 0 or labour < 0 or wastage < 0 or machine < 0 or selling < 0 or quantity <= 0:
        print("Invalid input")
        continue

    cost = (material + labour + wastage + machine) * quantity
    revenue = selling * quantity
    profit = revenue - cost
    percentage = (profit / cost) * 100 if cost > 0 else 0

    print("\nProduct:", name)
    print("Total Production Cost:", round(cost, 2))
    print("Total Revenue:", round(revenue, 2))
    print("Profit/Loss:", round(profit, 2))
    print("Profit Percentage:", round(percentage, 2), "%")

    if percentage >= target:
        print("Target achieved")
```

```
    else:
        print("Target not achieved")
```

**4. Output**

Sample Output:
Number of products: 2
Target profit %: 20
Product name: Table
Material cost: 500
Labour cost: 200
Wastage cost: 50
Machine cost: 100
Selling price: 1200
Quantity: 2
Table Cost: 1700.0 Revenue: 2400.0
Profit/Loss: 700.0
Profit %: 41.18
Target achieved
Product name: Chair
Material cost: 300
Labour cost: 150
Wastage cost: 50
Machine cost: 100
Selling price: 500
Quantity: 2
Chair Cost: 1200.0 Revenue: 1000.0
Profit/Loss: -200.0
Profit %: -16.67
Target not achieved

**5. Conclusion**

The Python program successfully evaluates the cost and profitability of multiple products by considering production expenses, selling prices, and quantities. It calculates total production cost, revenue, profit or loss, and profit percentage, and compares the results with the specified target profit percentage. This program helps businesses analyse product profitability and make informed pricing and production decisions.

# PROGRAM 9

**1. Problem Statement**

To develop a Python program to calculate and manage travel expenses incurred in different categories using currency exchange rates and conversion fees. The program converts expense amounts into Indian rupees, calculates the applicable conversion fees, determines the final expense for each category, and compares the total expenses with the available travel budget. It also identifies the category with the highest total expenditure.

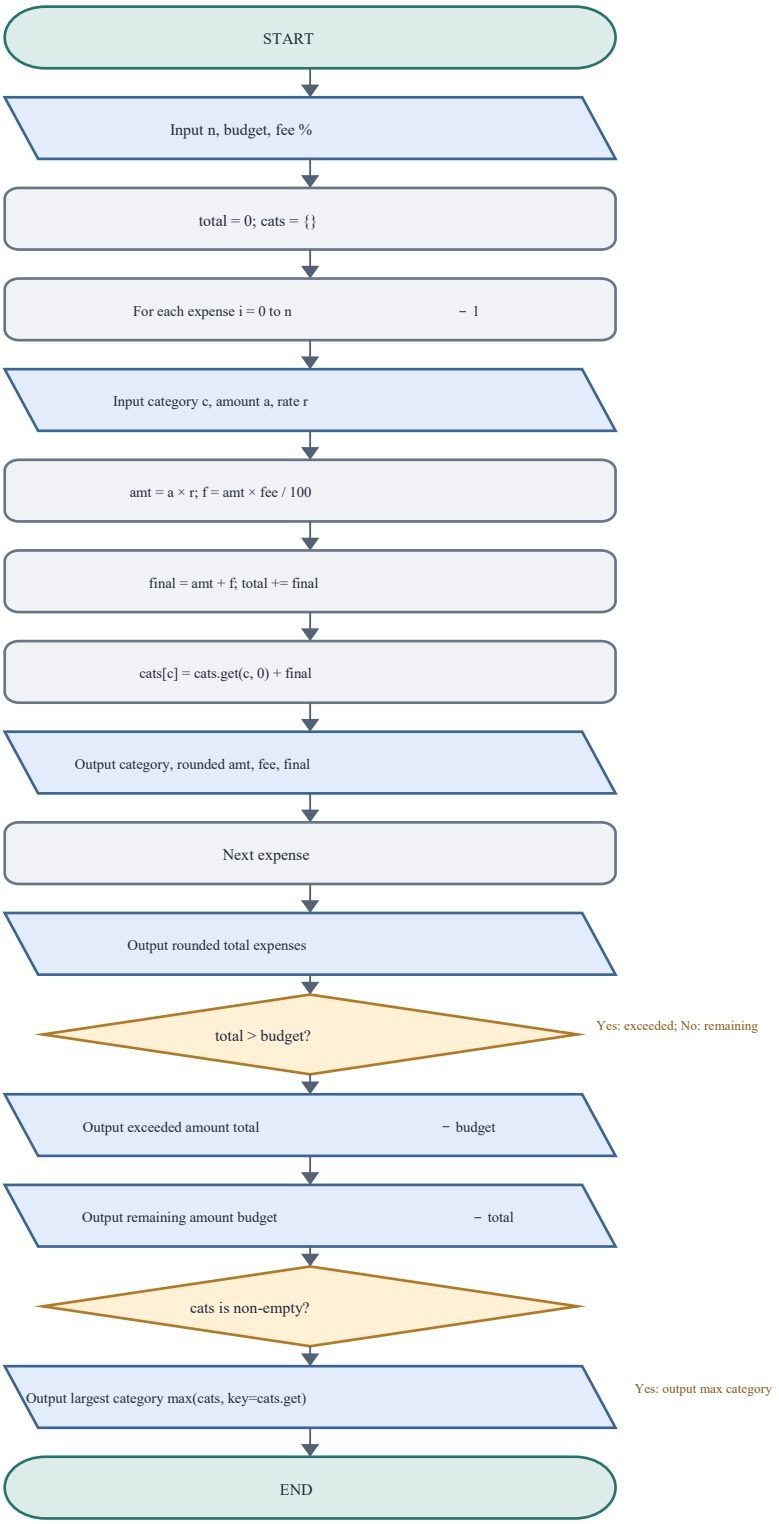
**2. Objectives**

1. To accept the number of travel expenses, total travel budget, and currency conversion fee percentage.
2. To input the expense category, expense amount, and applicable exchange rate for each expense.
3. To convert each expense amount into Indian rupees using the specified exchange rate.
4. To calculate the currency conversion fee for each expense.
5. To determine the final expense by adding the conversion fee to the converted amount.
6. To calculate the total expenses across all categories.
7. To compare the total expenses with the available travel budget.
8. To display the amount by which the budget is exceeded or the remaining budget.
9. To identify and display the category with the highest total expenditure.

**3. Assumptions**

1. The number of expenses is entered as a non-negative integer.
2. The travel budget and conversion fee percentage are entered as numerical values.
3. Each expense has a valid category, expense amount, and exchange rate.
4. The expense amount and exchange rate are non-negative numerical values.
5. The exchange rate represents the value of one unit of foreign currency in Indian rupees.
6. The converted amount is calculated by multiplying the expense amount by the exchange rate.
7. The conversion fee is calculated as a percentage of the converted amount.
8. The final expense is calculated by adding the conversion fee to the converted amount.
9. Expenses belonging to the same category are combined to calculate the category-wise total.
10. The total expenses are compared with the travel budget to determine whether the budget is exceeded or how much remains.
11. The category with the highest combined expenditure is identified as the largest expense category.
12. All monetary amounts displayed by the program are rounded to two decimal places.

# FLOW-CHART


START
Input n, budget, fee %
total = 0; cats = {}
For each expense i = 0 to n − 1
Input category c, amount a, rate r
amt = a × r; f = amt × fee / 100
final = amt + f; total += final
cats[c] = cats.get(c, 0) + final
Output category, rounded amt, fee, final
Next expense
Output rounded total expenses
total > budget?
Yes: exceeded; No: remaining
Output exceeded amount total − budget
Output remaining amount budget − total
cats is non-empty?
Output largest category max(cats, key=cats.get)
Yes: output max category
END

# *Pseudo-Code*

```
N = INT(INPUT("ENTER NUMBER OF EXPENSES: "))
BUDGET = FLOAT(INPUT("ENTER TOTAL TRAVEL BUDGET: "))
FEE_RATE = FLOAT(INPUT("ENTER CONVERSION FEE PERCENTAGE: "))
TOTAL = 0
CATEGORIES = {}

FOR I IN RANGE(N):
    CATEGORY = INPUT("ENTER EXPENSE CATEGORY: ")
    AMOUNT = FLOAT(INPUT("ENTER EXPENSE AMOUNT: "))
    RATE = FLOAT(INPUT("ENTER EXCHANGE RATE TO INR: "))

    CONVERTED = AMOUNT * RATE
    FEE = CONVERTED * FEE_RATE / 100
    FINAL = CONVERTED + FEE
    TOTAL += FINAL

    IF CATEGORY IN CATEGORIES:
        CATEGORIES[CATEGORY] += FINAL
    ELSE:
        CATEGORIES[CATEGORY] = FINAL

    PRINT("CATEGORY:", CATEGORY)
    PRINT("CONVERTED AMOUNT: RS.", ROUND(CONVERTED, 2))
    PRINT("CONVERSION FEE: RS.", ROUND(FEE, 2))
    PRINT("FINAL EXPENSE: RS.", ROUND(FINAL, 2))

PRINT("\NTOTAL TRAVEL EXPENSES: RS.", ROUND(TOTAL, 2))
PRINT("TRAVEL BUDGET: RS.", ROUND(BUDGET, 2))

IF TOTAL > BUDGET:
    PRINT("BUDGET EXCEEDED BY RS.", ROUND(TOTAL - BUDGET, 2))
ELSE:
    PRINT("BUDGET REMAINING: RS.", ROUND(BUDGET - TOTAL, 2))

PRINT("LARGEST EXPENSE CATEGORY:", MAX(CATEGORIES, KEY=CATEGORIES.GET))
```

# *Algorithm*

1. START.
2. ENTER THE NUMBER OF EXPENSES, TRAVEL BUDGET, AND CONVERSION FEE PERCENTAGE.
3. ENTER THE CATEGORY, AMOUNT, AND EXCHANGE RATE FOR EACH EXPENSE.
4. CONVERT EACH EXPENSE INTO INDIAN RUPEES.
5. CALCULATE THE CONVERSION FEE.
6. CALCULATE THE FINAL EXPENSE AND TOTAL EXPENSES.
7. FIND THE LARGEST EXPENSE CATEGORY.
8. COMPARE TOTAL EXPENSES WITH THE TRAVEL BUDGET.
9. DISPLAY THE TOTAL EXPENSES AND BUDGET STATUS.
10. STOP.

# *Code*

```
n = int(input("Enter number of expenses: "))
budget = float(input("Enter total travel budget: "))
fee_rate = float(input("Enter conversion fee percentage: "))
total = 0
categories = {}

for i in range(n):
    category = input("Enter expense category: ")
    amount = float(input("Enter expense amount: "))
    rate = float(input("Enter exchange rate to INR: "))

    converted = amount * rate
    fee = converted * fee_rate / 100
    final = converted + fee
    total += final
    if category in categories:
        categories[category] += final
    else:
        categories[category] = final
    print("Category:", category)
    print("Converted Amount: Rs.", round(converted, 2))
    print("Conversion Fee: Rs.", round(fee, 2))
    print("Final Expense: Rs.", round(final, 2))

print("\nTotal Travel Expenses: Rs.", round(total, 2))
print("Travel Budget: Rs.", round(budget, 2))

if total > budget:
    print("Budget exceeded by Rs.", round(total - budget, 2))
else:
    print("Budget remaining: Rs.", round(budget - total, 2))

print("Largest Expense Category:", max(categories, key=categories.get))
```

## 4. Output

Sample Output:
Number of expenses: 3
Travel budget: 20000
Conversion fee %: 2
Category: Accommodation
Expense amount: 100
Exchange rate: 85
Accommodation Converted: 8500.0
Fee: 170.0 Final: 8670.0
Category: Food
Expense amount: 50
Exchange rate: 85
Food Converted: 4250.0
Fee: 85.0 Final: 4335.0
Category: Shopping
Expense amount: 80
Exchange rate: 85
Shopping Converted: 6800.0
Fee: 136.0 Final: 6936.0
Total expenses: Rs. 19941.0
Budget remaining: 59.0
Largest category: Accommodation

## 5. Conclusion

The Python program successfully calculates travel expenses by converting foreign currency amounts into Indian rupees and applying the specified conversion fees. It calculates the total expenditure, compares it with the available travel budget, and identifies the category with the highest spending. This program helps travellers monitor their expenses, manage their budgets effectively, and make informed financial decisions during their trips.

# *PROGRAM 10*

1. Problem Statement

To develop a Python program to calculate the total cost, expected income, and financial feasibility of organizing an event based on venue expenses, equipment costs, catering charges, transport costs, certificate expenses, sponsorship, registration fees, and the expected number of participants. The program calculates the financial shortfall, determines the minimum number of registrations required to cover the fixed costs after sponsorship, and checks whether the event is financially feasible.

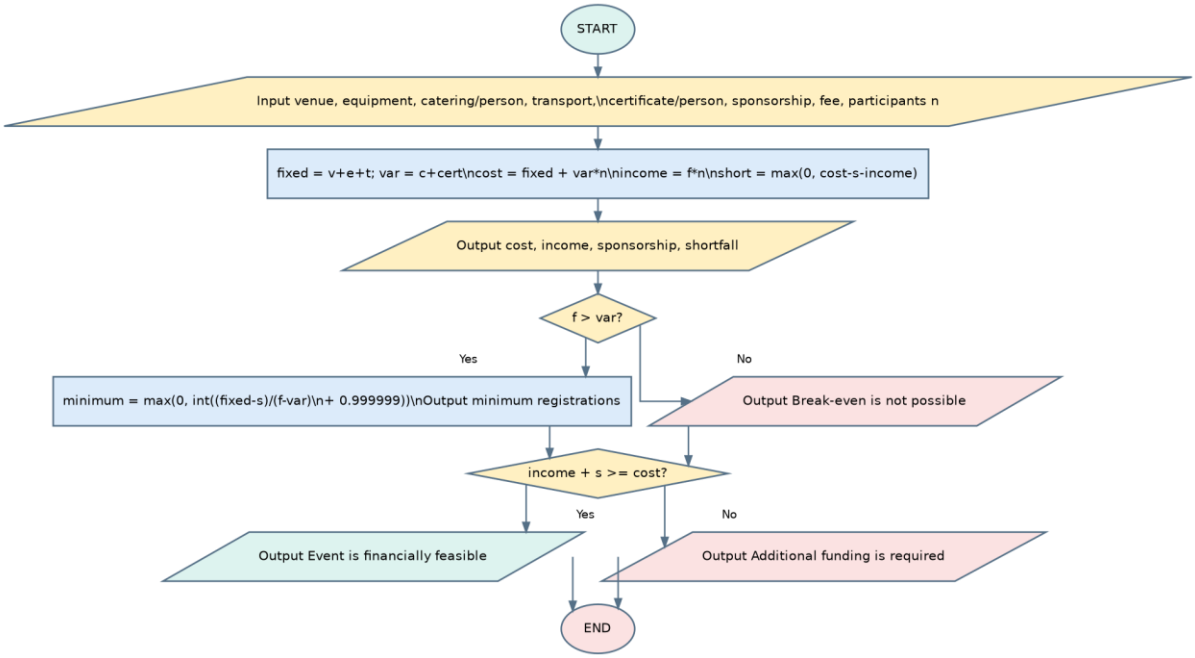
2. Objectives

1. To accept the various costs associated with organizing an event.
2. To calculate the total fixed cost, including venue, equipment, and transport expenses.
3. To calculate the variable cost per participant based on catering and certificate expenses.
4. To determine the total event cost based on fixed costs and the expected number of participants.
5. To calculate the total income generated through participant registration fees.
6. To consider sponsorship as an additional source of funding.
7. To calculate the financial shortfall after accounting for registration income and sponsorship.
8. To determine the minimum number of registrations required to cover the fixed costs after sponsorship, when possible.
9. To identify whether the event is financially feasible or requires additional funding.

3. Assumptions

1. All costs, registration fees, and sponsorship amounts are entered in Indian rupees.
2. The expected number of participants is entered as a non-negative integer.
3. Venue, equipment, and transport costs are treated as fixed costs.
4. Catering and certificate expenses are treated as variable costs per participant.
5. Total event cost is calculated by adding fixed costs to the variable cost per participant multiplied by the expected number of participants.
6. Total participant income is calculated by multiplying the registration fee by the expected number of participants.
7. Sponsorship is considered an additional source of funding for the event.
8. The financial shortfall is calculated as the amount by which the total event cost exceeds the combined registration income and sponsorship. If the combined funding is sufficient, the shortfall is zero.
9. Break-even registration calculations are performed only when the registration fee is greater than the variable cost per participant.
10. The event is considered financially feasible when registration income plus sponsorship is greater than or equal to the total event cost.
11. If the registration fee is less than or equal to the variable cost per participant, the program displays that break-even is not possible under its calculation method.

## *FLOW-CHART*




# *Pseudo-Code*

## 1. Pseudocode

```
START
INPUT venue v, equipment e, catering/person c, transport t,
certificate/person cert, sponsorship s, fee f, people n
fixed = v + e + t
var = c + cert
cost = fixed + var * n
income = f * n
short = max(0, cost - s - income)
OUTPUT total cost, income, sponsorship, shortfall
IF f > var:
minimum = max(0, int((fixed-s)/(f-var) + 0.999999))
OUTPUT minimum registrations
ELSE: OUTPUT 'Break-even is not possible'
IF income + s >= cost:
OUTPUT 'Event is financially feasible'
ELSE: OUTPUT 'Additional funding is required'
END
```

## *Algorithm*

1. Start.
2. Enter all event costs and the sponsorship amount.
3. Enter the registration fee and expected number of participants.
4. Calculate the fixed cost and variable cost.
5. Calculate the total event cost.
6. Calculate participant contributions and funding shortfall.
7. Calculate the minimum registrations required for break-even.
8. Check whether the event is financially feasible.
9. Display all results.
10. Stop.

## ***Code***

```
venue = float(input("Enter venue cost: "))
equipment = float(input("Enter equipment cost: "))
catering = float(input("Enter catering cost per participant: "))
transport = float(input("Enter transport cost: "))
certificate = float(input("Enter certificate cost per participant: "))
sponsorship = float(input("Enter sponsorship amount: "))
fee = float(input("Enter registration fee per participant: "))
participants = int(input("Enter expected participants: "))

fixed_cost = venue + equipment + transport
variable_cost = catering + certificate
total_cost = fixed_cost + variable_cost * participants
contribution = fee * participants
shortfall = max(0, total_cost - sponsorship - contribution)

if fee > variable_cost:
   minimum = max(0, int((fixed_cost - sponsorship) / (fee - variable_cost)) + 1)
   if (fixed_cost - sponsorship) % (fee - variable_cost) == 0:
      minimum -= 1
else:
   minimum = -1

print("\nTotal Event Cost:", total_cost)
print("Participant Contributions:", contribution)
print("Sponsorship Amount:", sponsorship)
print("Funding Shortfall:", shortfall)

if minimum == -1:
```

```
    print("Break-even is not possible with this registration fee")
else:
    print("Minimum Registrations for Break-even:", minimum)

if contribution + sponsorship >= total_cost:
    print("Event is financially feasible")
else:
    print("Additional funding is required")
```

**4. Output**

Sample Output:
Venue cost: 10000
Equipment cost: 5000
Catering per person: 200
Transport cost: 3000
Certificate per person: 50
Sponsorship: 5000
Registration fee: 400
Expected participants: 100
Total cost: 40000.0
Participant income: 40000.0
Sponsorship: 5000.0
Shortfall: 0
Minimum registrations: 44
Event is financially feasible

**5. Conclusion**

The Python program successfully evaluates the financial requirements of organizing an event by calculating fixed costs, variable costs, registration income, sponsorship, and financial shortfall. It determines the minimum registrations required to cover the fixed costs after sponsorship when the registration fee exceeds the variable cost per participant. The program also evaluates whether the event is financially feasible, helping organizers plan budgets, estimate funding requirements, and make informed financial decisions.